

Periodic Atomic-Density Visualization Specification

Plot-D3 normalized occupancy fields and browser-stable probability-mass meshes

mdstats

Contents

Purpose and status	1
Motive	2
Normative ownership	2
Public modules and exports	2
Input data and constraints	3
Frame collection	3
Framework topology	3
Selection	3
Frames and weights	3
Registered coordinates	3
Material mode	3
Framework-registered mode	3
Laboratory mode	4
Atomic occupancy measure	4
Periodic grid and cloud-in-cell deposition	4
Constant-interval default	4
Cloud-in-cell assignment	4
Periodic Gaussian smoothing	5
Grid-kernel ratio default	5
Spread-aware adaptive refinement	5
Resource-bounded refinement	6
Numerical options	6
Stored scalar field	7
Highest-density probability-mass shells	7
Downstream display-complexity boundary	7
Periodic rendering	7
Resource model	8
Progress reporting	8
Failure semantics	8
Numerical properties	8
Limitations and deferred work	8
Public example	9
Focused validation	9
External methods and references	10
Euclidean metric and density resolution	10

Purpose and status

This document is the normative specification for Plot-D3, introduced in mdstats 0.19.31a0. Plot-D3 constructs normalized time- or ensemble-averaged atomic occupancy fields in the same registered coordinate system used by the Plot-D1/D2 mean framework and trajectory overlays. It then renders nested highest-density probability-mass shells in the existing Plotly 3-D viewer. Version 0.19.34a0 replaces browser-side scalar-field triangulation with explicit periodic meshes prepared before HTML serialization. Version 0.19.38a0 replaces the fixed default grid shape with a cell-size-aware constant-interval policy. Version 0.19.39a0 couples the default Gaussian width to the realized grid interval and adds periodic spread-aware adaptive refinement with explicit resource-limit diagnostics.

The implemented pipeline is

registered atomic samples → periodic cloud-in-cell mass grid → normalized Gaussian smoothing → probability-mass meshes .

The scientific scalar field is prepared without importing Plotly. Rendering is an optional terminal operation and does not define the density, normalization, or source identity.

Motive

A trajectory line is effective for a few mobile atoms but becomes visually congested for many atoms or long trajectories. A time- or ensemble-averaged occupancy field exposes localization, split sites, annular motion, anisotropic vibration, and multimodal hopping without imposing a Gaussian displacement model.

The mean framework remains visible as a structural cue. Because both the framework and density use the Plot-D1 registration boundary, a density lobe is interpreted relative to the same periodic topology and display cell.

Normative ownership

Plot-D3 owns:

- atom-index and species selection for atomic occupancy fields;
- reuse of the Plot-D1 material, laboratory, and framework-registered gauges;
- a periodic fractional voxel grid;
- trilinear cloud-in-cell deposition;
- normalized Cartesian-isotropic Gaussian smoothing in reciprocal space;
- density normalization and physical units;
- highest-density probability-mass thresholds;
- seam-closed periodic isosurface coordinates;
- optional sparse raw-sample rendering;
- density-specific resource preflight;
- composition with framework and trajectory traces.

It does not own:

- framework connectivity or topology;
- atom identity across inconsistent collections;
- framework vertex or edge-length densities, which belong to Plot-D4;
- ring-local projections or site assignments;
- covariance ellipsoids;
- adaptive or nonuniform meshes;
- nonperiodic convolution;
- kinetic-state or hopping analysis.

Public modules and exports

mdstats/plotting/atomic_density.py
mdstats/plotting/framework_dynamics.py

The package root and mdstats.plotting export:

AtomicDensitySelection
AtomicDensityOptions
AtomicDensity3DRenderOptions
PeriodicScalarField3D

The existing scene functions are extended:

```
prepare_framework_dynamics_scene(  
    ...,  
    atomic_density_selections=...,  
    atomic_density_options=...,  
)
```

```
plot_framework_dynamics_3d(
    ...,
    density_options=...,
)
```

Input data and constraints

Frame collection

collection must be an AtomisticFrameCollection with fixed atom ordering and atomic numbers. Both trajectory and independent ensemble semantics are accepted. A density field does not require a time axis.

The first backend requires

```
np.all(collection.pbc)
```

because smoothing is a fully periodic convolution. Mixed and nonperiodic boundaries raise GraphAdapterError rather than silently wrapping a nonperiodic axis.

Framework topology

A valid FrameworkTopology remains required because the scene owns a registered mean framework. Every selected frame must preserve the framework graph identity and normalized periodic winding as specified by Plot-D1/D2.

Selection

AtomicDensitySelection forms the union of:

- explicit atom_indices;
- one or more chemical species selectors.

The resolved selection must be nonempty and lie within the collection. One field is created per selection. Each field records the exact selected atom indices.

Frames and weights

Plot-D3 uses the scene frame sequence and its normalized weights

$$\sum_t w_t = 1.$$

The current scene implementation uses uniform weights,

$$w_t = \frac{1}{N_f}.$$

Explicit time-quadrature and externally supplied ensemble weights remain future extensions.

Registered coordinates

Let H_d be the Plot-D1 display cell. For selected atom i in frame t , let $\mathbf{f}_i(t)$ be the reader-supplied fractional coordinate.

Material mode

$$\mathbf{u}_i(t) = \mathbf{f}_i(t).$$

Homogeneous cell deformation is removed because all samples are expressed in H_d .

Framework-registered mode

With framework drift $\Delta \mathbf{c}(t)$ from Plot-D1,

$$\mathbf{u}_i(t) = \mathbf{f}_i(t) - \Delta \mathbf{c}(t).$$

This removes a common framework translation from both the mean graph and atomic occupancy.

Laboratory mode

Instantaneous Cartesian positions are mapped back into the display-cell basis:

$$\begin{aligned}\mathbf{x}_i^{\text{lab}}(t) &= \mathbf{f}_i(t)H_t, \\ \mathbf{u}_i(t) &= \mathbf{x}_i^{\text{lab}}(t)H_d^{-1}.\end{aligned}$$

This retains homogeneous cell deformation while still producing one renderable periodic field.

For every periodic axis, samples are folded by

$$\tilde{\mathbf{u}}_i(t) = \mathbf{u}_i(t) - \lfloor \mathbf{u}_i(t) \rfloor.$$

Integer wrapping therefore leaves the field invariant.

Atomic occupancy measure

For selected atom set S , define

$$\rho_A(\mathbf{x}) = \sum_t w_t \sum_{i \in S} [\mathbf{x} - \tilde{\mathbf{u}}_i(t)H_d].$$

Its total measure is

$$\int_{\Omega} \rho_A(\mathbf{x}) d^3x = |S|,$$

where Ω is one display-cell volume. Thus:

- an individual-atom field integrates to one;
- a species field integrates to the number of selected atoms;
- the field has units of $\text{\AA}^{-3}$.

This is an occupancy density, not a mass or charge density.

Periodic grid and cloud-in-cell deposition

Constant-interval default

Let the display-cell vectors be the rows $\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3$ of H_d . When `grid_shape=None`, the default target lattice-grid interval is

$$h = 0.20 \text{ \AA}.$$

The resolved grid shape is

$$N_i = \max\left(1, \left\lceil \frac{\|\mathbf{a}_i\|}{h} \right\rceil\right).$$

Therefore the realized Euclidean edge interval along each fractional-grid axis is

$$h_i = \frac{\|\mathbf{a}_i\|}{N_i} \leq h.$$

This definition is valid for triclinic cells because it measures the Cartesian length of each lattice-grid edge. It does not assume orthogonal lattice vectors. For the Na-LTA primitive cell with three vectors of length about 17.363 angstrom, the default shape is (87, 87, 87) and the realized interval is about 0.1996 angstrom.

Supplying an explicit `grid_shape=(N_1, N_2, N_3)` overrides automatic sizing. The `grid_interval` value is then retained as configuration metadata but does not alter the explicit shape.

Cloud-in-cell assignment

Let the resolved grid shape be (N_1, N_2, N_3) . A folded fractional sample has scaled grid coordinate

$$\mathbf{y} = (N_1 u_1, N_2 u_2, N_3 u_3).$$

For each axis, write

$$y_a = b_a + d_a, \quad b_a = \lfloor y_a \rfloor, \quad 0 \leq d_a < 1.$$

The sample weight is distributed among the eight surrounding periodic grid vertices. For corner $\epsilon \in \{0, 1\}^3$,

$$W_\epsilon = \prod_{a=1}^3 \begin{cases} 1 - d_a, & \epsilon_a = 0, \\ d_a, & \epsilon_a = 1. \end{cases}$$

The weights satisfy

$$\sum_{\epsilon} W_\epsilon = 1,$$

so deposition preserves the exact discrete total measure before floating-point roundoff.

This trilinear particle-to-mesh assignment is the cloud-in-cell method adapted from Hockney and Eastwood [1]. The project-specific use here deposits an occupancy measure rather than charge or force density.

Periodic Gaussian smoothing

The deposited grid is convolved with a normalized Gaussian of Cartesian width σ :

$$K_\sigma(\mathbf{x}) = \frac{1}{(2\pi\sigma^2)^{3/2}} \exp\left(-\frac{|\mathbf{x}|^2}{2\sigma^2}\right).$$

Using the periodic convolution theorem, each reciprocal mode is multiplied by

$$\hat{K}_\sigma(\mathbf{k}) = \exp\left(-\frac{\sigma^2|\mathbf{k}|^2}{2}\right).$$

For integer fractional mode $\mathbf{m} \in \mathbb{Z}^3$, the Cartesian reciprocal vector is

$$\mathbf{k} = 2\pi H_d^{-1} \mathbf{m}.$$

The metric is therefore Cartesian-isotropic even for a triclinic display cell. The inverse FFT result is clipped only for tiny negative roundoff and is then renormalized to the exact selected-atom count.

Setting an explicit `gaussian_bandwidth=0` disables smoothing and returns the raw cloud-in-cell field.

The Gaussian kernel is a visualization regularizer. It does not assert that the underlying atomic displacement distribution is Gaussian.

Grid-kernel ratio default

When `gaussian_bandwidth=None`, the bandwidth is derived from the longest realized lattice-grid interval,

$$\sigma = r_g h_{\max}, \quad h_{\max} = \max_i h_i,$$

with default ratio

$$r_g = 2.$$

The ratio is applied after integer grid resolution, so the actual Gaussian is exactly twice the longest realized grid edge rather than merely twice the requested target interval.

Spread-aware adaptive refinement

For selected atom i , let $\bar{\mathbf{x}}_i$ be its periodic Frechet mean in the display-cell metric. Minimum-image displacements are

$$\Delta \mathbf{x}_i(t) = \text{MI}(\mathbf{G}_i(t) - \bar{\mathbf{x}}_i).$$

The scalar positional standard deviation used for comparison with an isotropic Gaussian is

$$s_i = \sqrt{\frac{1}{3} \sum_t w_t \|\Delta \mathbf{x}_i(t)\|^2} = \sqrt{\frac{\text{tr } C_i}{3}}.$$

This is a per-Cartesian-component scale. Convolution with an isotropic Gaussian adds σ^2 to each component variance. The field-level reference spread is the configured quantile of the per-atom values,

$$s_q = Q_q(s_i), \quad q = 0.10 \text{ by default}.$$

Automatic refinement is triggered when

$$\sigma_{\text{nominal}} > \alpha s_q,$$

where the default is $\alpha = 0.5$. The requested adaptive target is

$$\sigma_{\text{target}} = \alpha s_q, \quad h_{\text{target}} = \frac{\sigma_{\text{target}}}{r_g}.$$

At $\alpha = 0.5$, a single Gaussian broadening step increases a truly isotropic sample SD by at most

$$\frac{\sqrt{s^2 + \sigma^2}}{s} \leq \sqrt{1 + 0.5^2} \approx 1.118,$$

or about 12 percent.

The periodic mean uses the Frechet/Karcher center-of-mass construction on the flat torus [6,7]. The implementation iterates weighted Cartesian minimum-image vectors and therefore respects a triclinic Euclidean metric.

Resource-bounded refinement

A globally uniform three-dimensional grid can become prohibitively large for a very narrow atomic distribution. Adaptive refinement therefore obeys the per-field share of `max_density_voxels`:

1. the nominal grid must fit the budget;
2. the grid is refined toward h_{target} ;
3. if the target exceeds the budget, the finest admissible grid is selected;
4. a warning reports the unresolved σ/s_q ratio.

The code never silently claims that a budget-limited field reached the requested broadening criterion. Explicit `grid_shape` or explicit `gaussian_bandwidth` values are preserved and generate a warning rather than being changed automatically.

Numerical options

```
AtomicDensityOptions(
    grid_shape=None,           # automatic by default
    grid_interval=0.20,       # nominal target, angstrom
    gaussian_bandwidth=None,   # derive from grid ratio
    gaussian_to_grid_ratio=2.0,
    adaptive_smearing=True,
    max_smearing_to_sample_sd_ratio=0.50,
    sample_sd_quantile=0.10,
    store_sample_positions=False,
)
```

The public helper

```
resolve_density_grid_shape(
    display_cell,
    grid_shape=options.grid_shape,
    grid_interval=options.grid_interval,
)
```

returns the resolved integer grid shape. `density_grid_intervals(...)` returns the three realized lattice-grid edge lengths.

Stored scalar field

PeriodicScalarField3D stores:

```
field_key: str
label: str
values: ndarray[float64]          # (N1, N2, N3), in angstrom^-3
display_cell: ndarray[float64]    # (3, 3)
total_measure: float              # selected atom count
selected_atom_indices: tuple[int, ...]
gaussian_bandwidth: float
sample_positions: ndarray | None
metadata: Mapping[str, Any]
```

Derived properties include:

```
grid_shape
voxel_volume
integral
threshold_for_mass_fraction(q)
```

The field arrays are immutable. Rendering does not modify them.

Highest-density probability-mass shells

A density threshold ρ_q for requested mass fraction q is defined by

$$\int_{\tilde{\rho}(\mathbf{x}) \geq \rho_q} \tilde{\rho}(\mathbf{x}) d^3x \geq q \int_{\Omega} \tilde{\rho}(\mathbf{x}) d^3x,$$

using the smallest discrete superlevel set that reaches the requested mass. This follows the highest-density-region construction described by Hyndman [2].

The default shells enclose 50%, 80%, and 95% of the total field measure. These are more comparable across species and smoothing widths than thresholds defined as arbitrary fractions of the maximum voxel value.

The scientific thresholds are converted to explicit triangular shells before HTML serialization. The default renderer applies the topologically robust Lewiner marching-cubes implementation supplied by scikit-image [3,4] to a periodically seam-closed scalar grid. This avoids browser-side triangulation and does not alter the stored scientific density.

Downstream display-complexity boundary

This module owns scientific scalar fields, HDR mass fractions, and contour levels. It does not own final browser mesh complexity. Dense and sparse renderers convert a shell to DensityShellGeometry; density_mesh_contracts, density_scene_budget, density_mesh_simplify, density_scene_fit, and density_render_budget then own the display-only adaptation and exact export gate. No downstream fitting step may change the density estimate or HDR threshold.

Periodic rendering

The grid is seam-closed for rendering by copying the first grid plane to an additional endpoint at fractional coordinate one along each axis. The render grid therefore has shape

$$(N_1 + 1) \times (N_2 + 1) \times (N_3 + 1).$$

Cartesian coordinates are

$$\mathbf{x}_{abc} = \left(\frac{a}{N_1}, \frac{b}{N_2}, \frac{c}{N_3} \right) H_d.$$

One Plotly Mesh3d trace is created for each requested probability-mass shell. The vertices and triangle indices are already explicit when the HTML is written. Nested shells use one legend group and a common field color, with higher-density inner shells rendered more strongly than diffuse outer shells. Plotly therefore performs only interactive triangle rendering [5].

`render_mode="voxel_cloud"` provides a dependency-light and WebGL-conservative fallback. It draws a deterministic subset of voxel centers inside the outer highest-density region. When `store_sample_positions=True`, the scene may also render the folded raw samples as a separate diagnostic cloud.

Resource model

LD10 resolves one authoritative runtime budget for the complete framework-dynamics scene. By default it uses 80% of detected available memory, 90% of available CPUs, and a 1,200-second preparation-plus-rendering objective. Users may override maximum memory, threads, and wall time through `FrameworkDynamicsResources`, `MDSTATS_*` environment variables, or the example command line. Memory and thread requests remain bounded by the actual process/job allocation.

Density compatibility fields such as

```
max_density_fields
max_density_voxels
max_density_samples
max_density_render_points
max_density_traces
```

are resolved from that scene budget when omitted. They are not universal package constants. Explicit low-level values may only tighten the scene policy. Exact scene preflight checks sample arrays, dense or sparse retained fields, planning structures, kernel work, contour workspaces, output reserve, and worker-pool memory together. A calibrated input-independent work model also rejects scenes that are not expected to finish within the wall-time objective.

Rendering separately checks browser-output profiles after display replication. No automatic grid coarsening, field dropping, sample truncation, shell omission, or mesh-to-point substitution occurs. See `density_runtime_resource_policy_ld10_spec.{md,pdf}` for the normative formulas and override precedence.

Progress reporting

`prepare_atomic_density_fields(...)` accepts the package-wide keyword-only `progress=` port. Each requested density selection emits one structured `field_realization` item with its ordinal, field key, selected backend, logical grid, and resolved Gaussian width. The module remains silent when no port is supplied.

`progress_callback=Callable[[str], None]` is retained only as a deprecated compatibility alias. The normative event schema is defined in `docs/specs/progress_spec.{md,pdf}`.

Failure semantics

GraphAdapterError invalid selection, mixed periodicity, singular display cell, incompatible framework registration, malformed field, or destroyed normalization.

GraphComplexityError density field, voxel, sample, render-point, or trace resource limit exceeded.

GraphStyleError invalid grid shape, bandwidth, mass fractions, opacity, or marker size.

GraphVisualizationError optional Plotly import or HTML serialization failure.

Failures never change the selected atom set or silently discard frames.

Numerical properties

The implementation guarantees, to floating-point tolerance,

$$\int_{\Omega} \tilde{\rho}_A(\mathbf{x}) d^3x = |S|.$$

It also guarantees invariance under any integer shift of a selected atom's fractional coordinate. The result is not invariant under changing the display cell, registration mode, grid shape, or smoothing bandwidth; those choices are explicit provenance.

Limitations and deferred work

- Uniform frame weights only.
- Fully periodic cells only.

- Uniform fractional grids only.
- One global Gaussian bandwidth per prepared scene call.
- No adaptive bandwidth or atom-specific kernel.
- No covariance ellipsoids or principal-displacement glyphs.
- No rolling-window or difference-density fields.
- No ring-local coordinate projection.
- No Gaussian cube, VTK, or NumPy export helper beyond direct array access.
- Density is folded into one display cell; long-range displacement is represented by Plot-D2 trajectories, not by the occupancy field.

Plot-D4 will add framework vertex occupancy and framework edge-length density as separate fields because their units differ.

Public example

```
from mdstats import (
    AtomicDensity3DRenderOptions,
    AtomicDensityOptions,
    AtomicDensitySelection,
    FrameworkDynamicsOptions,
    prepare_framework_dynamics_scene,
    plot_framework_dynamics_3d,
)

scene = prepare_framework_dynamics_scene(
    collection,
    framework_topology,
    atomic_density_selections=(
        AtomicDensitySelection(species="Na",), label="Na occupancy"),
        AtomicDensitySelection(species="O",), label="O occupancy"),
    ),
    atomic_density_options=AtomicDensityOptions(
        grid_interval=0.20,
        gaussian_to_grid_ratio=2.0,
        adaptive_smearing=True,
    ),
    options=FrameworkDynamicsOptions(
        registration_mode="framework_registered",
    ),
)

rendered = plot_framework_dynamics_3d(
    scene,
    density_options=AtomicDensity3DRenderOptions(
        mass_fractions=(0.50, 0.80, 0.95),
    ),
)
rendered.write_html("framework_atomic_density.html")
```

Focused validation

The focused tests verify:

1. a one-atom field integrates to one;
2. a multi-species field integrates to the selected atom count;
3. integer periodic wrapping leaves the field unchanged;
4. Gaussian smoothing preserves normalization and reduces the peak;
5. independent ensembles are accepted;
6. framework registration removes common rigid drift;
7. material and laboratory modes differ under cell deformation;
8. highest-density thresholds are monotone;

9. field and voxel resource preflight is transactional;
10. mixed periodicity is rejected;
11. explicit triangular density meshes and optional sample clouds serialize to HTML;
12. render-point resource limits are enforced;
13. automatic grid sizing obeys the requested interval in cubic and triclinic cells;
14. explicit grid shapes override the interval policy;
15. resolved grid metadata records the target and realized intervals; and
16. Plot-D1/D2 and generic graph-renderer regressions remain unchanged.

External methods and references

The borrowed methods are explicitly separated from project-owned integration:

- Cloud-in-cell deposition is adapted from Hockney and Eastwood [1].
- Highest-density superlevel regions are adapted from Hyndman [2].
- Lorensen and Cline [3] and Lewiner et al. [4] supply the external surface-extraction theory;
- Plotly supplies only the optional interactive Mesh3d renderer [5].
- Registration, topology binding, density normalization, triclinic reciprocal metric construction, seam closure, resource accounting, and combined scene composition are project-specific implementations.

[1] R. W. Hockney and J. W. Eastwood, *Computer Simulation Using Particles*, Adam Hilger, 1988. Reprint DOI: 10.1201/9780367806934.

[2] R. J. Hyndman, “Computing and Graphing Highest Density Regions,” *The American Statistician* **50**, 120-126 (1996). DOI: 10.1080/00031305.1996.10474359.

[3] W. E. Lorensen and H. E. Cline, “Marching Cubes: A High Resolution 3D Surface Construction Algorithm,” *Computer Graphics* **21**, 163-169 (1987), DOI: 10.1145/37402.37422.

[4] T. Lewiner, H. Lopes, A. W. Vieira, and G. Tavares, “Efficient Implementation of Marching Cubes’ Cases with Topological Guarantees,” *Journal of Graphics Tools* **8**, 1-15 (2003), DOI: 10.1080/10867651.2003.10487582.

[5] Plotly Technologies Inc., `plotly.graph_objects.Mesh3d` API documentation, <https://plotly.com/python/3d-mesh/>.

Euclidean metric and density resolution

The Plotly scene preserves Cartesian Euclidean distance by choosing manual scene aspect ratios proportional to the final Cartesian axis ranges. Consequently, for axis range lengths R_x, R_y, R_z and scene aspect lengths A_x, A_y, A_z ,

$$\frac{A_x}{R_x} = \frac{A_y}{R_y} = \frac{A_z}{R_z}.$$

Thus one angstrom has the same displayed scale along every Cartesian axis.

The density field itself is sampled on a uniform fractional grid. In a highly oblique cell, an excessively narrow Gaussian relative to the real-space grid edge length can make a marching-cubes shell appear systematically elliptical although the scene metric is correct. Let

$$h_{\max} = \max_a \frac{\|\mathbf{a}_a\|}{N_a}, \quad r = \frac{\sigma}{h_{\max}}.$$

The implementation warns when $r < 1.5$. Version 0.19.39a0 uses a nominal `grid_interval=0.20` angstrom and derives $\sigma = 2h_{\max}$ by default. The grid shape is resolved separately for each display-cell vector, so numerical resolution scales with cell size. For the Na-LTA primitive cell, the nominal grid is 87^3 and the nominal Gaussian is approximately 0.399 angstrom. Spread-aware refinement may select a finer grid and smaller Gaussian, subject to the configured voxel budget.

[6] M. Frechet, “Les elements aleatoires de nature quelconque dans un espace distance,” *Annales de l’Institut Henri Poincare* **10**, 215-310 (1948).

[7] H. Karcher, “Riemannian center of mass and mollifier smoothing,” *Communications on Pure and Applied Mathematics* **30**, 509-541 (1977), DOI: 10.1002/cpa.3160300502.